

1. 왜 Word2Vec이 필요한가? (텍스트 → 숫자 벡터의 문제)

1-1. 자연어 처리에서 “벡터화” 문제

컴퓨터는 텍스트를 그대로 이해 못하고, **숫자 벡터**로 바꿔야만 다룰 수 있어요.
그래서 “단어 → 숫자” 변환이 필수입니다.

초기에는 보통 이런 방식들이 쓰였습니다.

- **One-hot 인코딩**

- 단어 사전 크기 = V 라 하면,
- 각 단어를 길이 V 짜리 벡터로 표현
- 해당 단어 위치만 1, 나머지는 전부 0
- 예: 단어 사전 = {"I", "like", "movie"}
- "I" → [1,0,0]
- "like" → [0,1,0]
- "movie" → [0,0,1]

- **문제점**

1. **희소(sparse)**: 거의 다 0 → 메모리 비효율적

2. **단어 사이 유사성이 전혀 반영 안 됨**

- "good"과 "nice"는 의미가 비슷한데, one-hot 상으로는 완전히 다른 벡터
- 벡터 간 유클리드 거리, 코사인 유사도가 의미를 반영하지 못함

그래서 나온 것이 **분산 표현(distributed representation)** 또는 **임베딩(embedding)** 이고,
그 대표적인 초기 방법이 **Word2Vec**입니다.

2. 분산 표현(Distributed Representation)과 임베딩

2-1. 분산 표현의 아이디어

- 단어를 **짧은, 밀집(dense)된 실수 벡터**로 표현

- 예: 임베딩 차원 = 100 이라면
 - "good" → [0.21, -0.11, 0.03, ...]
 - "nice" → [0.19, -0.09, 0.05, ...]
- 학습 데이터에서의 ****공동 출현 패턴(co-occurrence)****을 반영해서, 의미가 비슷한 단어끼리 비슷한 벡터가 되도록 학습

2-2. 분산 표현의 장점

1. 의미적 유사성 반영

- "king" - "man" + "woman" ≈ "queen" 같은 벡터 연산이 가능해짐

2. 차원 축소

- one-hot: 차원 = vocab size(수만~수십만)
- 임베딩: 차원 = 보통 수십~수백

3. 다양한 다운스트림 작업에 사용

- 감성 분석, 문서 분류, 개체명 인식 등에서 입력 특징으로 사용

이러한 분산 표현을 효율적으로 학습하는 대표적인 방법이 **Word2Vec**이고, 그 안에 **CBOW**와 **Skip-gram** 두 가지 모델 구조가 있습니다.

3. Word2Vec의 핵심 아이디어

Word2Vec은 "단어의 주변 단어를 예측하는 작업" 을 통해 임베딩을 학습합니다.

• 분포 가설(Distributional Hypothesis)

"비슷한 문맥에서 등장하는 단어들은 비슷한 의미를 가진다"

- 이를 수학적으로/알고리즘적으로 구현한 것이 Word2Vec입니다.

Word2Vec에는 두 가지 대표적인 구조가 있습니다.

1. CBOW (Continuous Bag-of-Words)

- "문맥(context) → 중심 단어(target)" 예측

2. Skip-gram

- "중심 단어 → 주변 단어" 예측

둘 다 아주 얇은 신경망(입력층-은닉층-출력층) 구조이며,
은닉층 가중치가 곧 단어 임베딩 행렬로 사용됩니다.

4. CBOW 모델: 주변 단어로 중앙 단어 예측

4-1. CBOW의 기본 구조

예시 문장:

"The cat sat on the mat"

- 윈도우 크기(window size) = 2라고 하면,
- "sat"를 중심 단어로 잡았을 때, 주변(context)은:
 - 왼쪽 2단어: "The", "cat"
 - 오른쪽 2단어: "on", "the"
 - $\rightarrow \text{context} = \{\text{"The"}, \text{"cat"}, \text{"on"}, \text{"the"}\}, \text{target} = \text{"sat"}$

입력

- 문맥 단어들을 one-hot 또는 임베딩으로 바꿔서 평균(or 합)을 구함
- Bag-of-Words라서 문맥 단어들의 순서는 무시

은닉층

- 임베딩 차원 d 라고 할 때, 각각의 단어를 d -차원 벡터로 매핑
- 문맥 단어 임베딩의 평균 $\rightarrow$ 문맥 벡터 h

출력층

- 전체 단어 사전 크기 = V 라 하면,
- 소프트맥스(softmax)를 통해 "각 단어가 타겟이 될 확률"을 출력
- 우리가 진짜 타겟으로 삼은 중심 단어의 확률을 크게 만들도록 학습

4-2. CBOW의 특징

- 문맥 $\rightarrow$ 단어 예측
- 주변 단어들을 모두 고려해서 target 단어를 맞추려고 하므로

- 전체 문맥 정보를 “모아서” 표현
- 큰 말뭉치에서 학습할 때 빠르고 안정적이라는 장점이 있음

5. Skip-gram 모델: 중앙 단어로 주변 단어 예측

5-1. Skip-gram의 기본 구조

같은 문장: “The cat sat on the mat”

중심 단어: “sat”, window size = 2

Skip-gram은 반대로:

- 입력: 중심 단어 “sat”
- 출력: 각 주변 단어 “The”, “cat”, “on”, “the”를 하나씩 예측하는 여러 개의 학습 샘플을 만듦.
 - (sat → The), (sat → cat), (sat → on), (sat → the)

입력

- 중심 단어를 one-hot → 임베딩 벡터 v_{sat}

출력

- 각 context 단어가 될 확률을 softmax로 출력
- 실제 context 단어가 되도록 확률을 높이는 방향으로 학습

5-2. Skip-gram의 특징

- 중심 단어 한 개로부터 여러 개의 문맥 단어를 예측
- 희귀 단어 처리에 특히 유리
 - 중심 단어가 희귀 단어여도, 주변에 자주 나오는 단어들을 보고 임베딩을 잘 학습
- 일반적으로 **Skip-gram이 CBOW보다 성능이 좋은 경우가 많지만**, CBOW가 더 빠르게 학습되는 경우도 있음.

6. Word2Vec 학습을 효율적으로 만드는 기법들

원래 이 모델은 출력이 vocab size V 짜리 softmax인데,
대부분의 말뭉치는 V 가 매우 크기 때문에 계산량이 어마어마합니다.

그래서 **Word2Vec 논문에서는 두 가지 주요 기법을 제안합니다.**

6-1. Negative Sampling

- 각 학습 샘플마다,
"정답 단어(target)" 1개와,
"무작위로 뽑은 샘플(negative word)" 몇 개만 두고 binary classification 을 함.
- 예:
 - 실제 (sat → on)이면,
 - positive: (sat, on, label=1)
 - negative: (sat, car, label=0), (sat, music, label=0), ...
- 이렇게 하면:
 - 전체 softmax 대신 **로지스틱 회귀 형태의 이진 분류들만** 계산하면 되므로 훨씬 빠름
 - 특히 Skip-gram + Negative Sampling(SGNS)이 많이 쓰이는 조합

6-2. Subsampling (빈도 높은 단어 제거)

- "the", "a", "of" 같은 매우 자주 등장하는 단어는 정보량이 적음
- 이런 단어들은 학습에서 의도적으로 자주 버림(확률적으로)
- 이렇게 하면
 - 전체 데이터 크기 감소 → 학습 속도 증가
 - 자주 등장하는 단어가 학습을 "잡아먹는 것"을 막고
좀 더 의미 있는 단어들에 집중 가능

7. Gensim으로 Word2Vec 구현 (실습 관점 개념 정리)

책에서 나오는 **Gensim**은 파이썬에서 Word2Vec을 간단하게 돌려볼 수 있게 해주는 라이브러리입니다.

Word2Vec을 Gensim으로 학습하는 과정은 개념적으로 이런 순서예요:

1. 코퍼스 준비

- 문장을 토큰화(tokenize)해서 단어 리스트의 리스트 형태로 준비
예:
- sentences = [
 - ["i", "love", "nlp"],
 - ["nlp", "is", "fun"],
 - ...
-]

2. 모델 정의

- CBOW vs Skip-gram 선택
 - sg=0 → CBOW
 - sg=1 → Skip-gram
- 벡터 차원(vector_size), window size, min_count(최소 등장 횟수) 등 설정

3. 학습

- model.train() 등을 통해 실제 코퍼스를 학습하면서,
단어 임베딩 행렬을 업데이트

4. 학습된 임베딩 사용

- model.wv["word"] → 해당 단어의 임베딩 벡터
- model.wv.most_similar("king") → 유사한 단어들 조회
- model.wv.similarity("man", "woman") → 유사도 수치

실제로 코드를 돌려보면,

- 의미가 비슷한 단어들이 가까운 위치에 모이고,
- 단어 간 유사도를 코사인 유사도(cosine similarity)로 측정할 수 있다는 걸 확인하게 됩니다.

8. 옛날 신경망 언어모델들: NNLM, RNN, LSTM, CNN 등

6장에서는 Word2Vec뿐 아니라,

Transformer 이전에 쓰이던 신경망 언어 모델들도 간단히 소개하는 경우가 많아요.

8-1. NNLM (Neural Network Language Model)

- Bengio(2003)의 초기 신경망 언어 모델
- 아이디어:
 1. 각 단어를 임베딩으로 바꿈
 2. 과거 $n - 1$ 개 단어의 임베딩을 concat해서
 3. MLP(다층 퍼셉트론)에 넣어 다음 단어 확률 분포를 예측
- n-gram 언어모델의 한계를 딥러닝으로 보완한 초기 시도

8-2. RNN, LSTM

- **RNN (Recurrent Neural Network)**
 - 시퀀스 데이터를 처리하는 기본 구조
 - 이전 시점의 hidden state와 현재 입력을 결합해 다음 hidden state를 계산
→ 문장 전체의 문맥 정보를 순차적으로 반영 가능
 - 하지만 “장기 의존성(long-term dependency)” 문제로, 앞쪽 정보가 뒤로 갈수록 사라짐(Vanishing gradient)
- **LSTM (Long Short-Term Memory)**
 - RNN의 장기 의존성 문제를 해결하기 위해 제안된 구조
 - 입력 게이트(input gate), 망각 게이트(forget gate), 출력 게이트(output gate)를 통해
정보를 선택적으로 기억/망각
 - 기계번역, 언어모델링 등에서 오랫동안 SOTA를 찍었던 구조

8-3. CNN 기반 텍스트 모델

- CNN은 원래 이미지에서 필터를 씌우는 구조지만,
텍스트에서도 1D convolution으로 n-gram 특징을 뽑는 방식으로 사용됨
- 예:
 - filter size = 3이면, 3단어씩 묶어서 local pattern을 추출

- max pooling으로 중요한 특징만 뽑아서 문장 임베딩으로 사용
- 장점:
 - 병렬화 쉬움
 - 지역적인 패턴을 잘 포착

그래서 책에서 "CBOW/Skip-gram → RNN/LSTM/CNN → (나중에) Transformer" 같은 흐름으로

자연어 처리 모델의 발전 과정을 소개하는 경우가 많아요.

9. 옵티마이저 관점: Adam vs 이전 최적화 방법

Word2Vec 자체는 크게 옵티마이저 이야기를 강조하지 않지만, "딥러닝 모델 학습"이라는 관점에서 **Adam**은 거의 표준처럼 쓰이는 옵티마이저라서 같이 언급됩니다.

9-1. 기존 옵티마이저: SGD, Momentum

- **SGD (Stochastic Gradient Descent)**
 - 파라미터 θ 를 다음과 같이 업데이트:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$
 - 고정된 학습률 η 사용
 - 단순하지만, 학습률 튜닝이 민감하고 수렴이 느림
- **Momentum**
 - "관성" 개념 도입
 - 이전 업데이트 방향을 부분적으로 유지함으로써, 계곡과 같은 지형에서 더 빠르게 진행
 - 수식:

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla L(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

9-2. Adam(Adaptive Moment Estimation)

- **Adam**은
 - **Momentum + Adagrad(적응형 학습률)** 를 합쳐놓은 느낌
- 핵심 아이디어:
 - 1차 모멘트(평균)와 2차 모멘트(제곱 평균)를 추적해서, 각각의 파라미터마다 자동으로 학습률을 조정
- 장점:
 - 학습률에 덜 민감
 - 초기 설정만 잘하면 대부분의 네트워크에서 “그냥 잘 동작”
 - 특히 RNN/LSTM, CNN, Transformer 등 복잡한 모델에서 자주 사용

6장 리뷰에서 언급한 것처럼,

- Adam이 왜 **SGD나 Momentum, Adagrad**보다 나은지 비교해주면
- “왜 우리가 요즘 대부분 Adam을 쓰는지” 이해하는 데 도움이 됩니다.

10. Word2Vec 이후: 더 발전한 임베딩들 한 스푼

6장에서는 깊이 안 들어가더라도, 보통 다음과 같은 흐름을 간단히 언급합니다.

1. **Word2Vec, GloVe**: 정적인 단어 임베딩
 - 단어 하나당 벡터 하나 (문맥에 따라 바뀌지 않음)
 - “bank(은행)” vs “bank(강둑)” 같은 동음이의어 처리에 한계
2. **Contextual Embeddings (ELMo, BERT, GPT, ...)**
 - 문맥에 따라 임베딩이 달라지는 구조
 - Transformer 기반 모델들이 대부분 이런 방식

6장은 주로 Word2Vec 중심이지만,

“이후의 Transformer 기반 임베딩은 Word2Vec의 아이디어를 더 확장한 것”

정도로 연결해둔다고 생각하면 흐름 잡는 데 좋습니다.